

Name: vrajesh

Authentication and Access Control

1. Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

Answer:

Python Code

```
```python  
Mock Zomato Admin Login

admin_username = "admin"
admin_password = "zomato123"

username = input("Enter Username: ")
password = input("Enter Password: ")

if username == admin_username and password ==
 admin_password:
 print("Access Granted")
 else:
 print("Access Denied")
```
```

Explanation:

- * The program stores the correct username and password in variables.*
- * It asks the user to enter a username and password.*
- * If both match the stored credentials, it displays ****"Access Granted"****.*
- * Otherwise, it displays ****"Access Denied"****.*

2. Write a Python function `check_access(user_role, resource)` that returns `True` if the `user_role` is allowed to access the resource, otherwise `False`. Test it with roles like `'customer'`, `'restaurant_owner'`, and resources like `'menu_edit'`, `'order_history'`, and `'admin_dashboard'`.

*****Hint:** Use a dictionary to map roles to allowed resources.***

Answer:

Python Code

```
```python  
def check_access(user_role, resource):
 permissions = {
 "customer": ["order_history"],
 "restaurant_owner": ["menu_edit",
 "order_history"],
 "admin": ["menu_edit", "order_history",
 "admin_dashboard"]
 }
```

```
return resource in permissions.get(user_role, [])
```

```
Test Cases
```

```
print(check_access("customer", "order_history")) #
 True
```

```
print(check_access("customer", "menu_edit")) #
 False
```

```
print(check_access("restaurant_owner", "menu_edit")) # True
```

```
print(check_access("restaurant_owner",
 "admin_dashboard")) # False
```

```
print(check_access("admin", "admin_dashboard"))
 # True
```

```
...
```

```
Explanation:
```

- \* A dictionary stores which resources each role can access.*
- \* The function checks whether the requested resource is available for that role.*
- \* It returns **\*\*True\*\*** if access is allowed and **\*\*False\*\*** otherwise.*

---

*## 3. Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.*

*#### Answer:*

*#### Python Code*

```
``python
import random

correct_password = "paytm123"

password = input("Enter Password: ")

if password == correct_password:

otp = random.randint(100000, 999999)

print("Generated OTP:", otp)

user_otp = int(input("Enter OTP: "))
```

```
if user_otp == otp:
 print("Login Successful")
 else:
 print("Invalid OTP")

else:
 print("Incorrect Password")
 ...
```

*### Explanation:*

- \* The user first enters the password.*
- \* If the password is correct, a random 6-digit OTP is generated.*
- \* The user enters the OTP.*
- \* If the OTP matches, the login is successful.*



---

*## 4. Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days.*

*\*\*Constraint:\*\* Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.*

*### Answer:*

*### Python Code*

*```python*

*from datetime import datetime*

```
today = datetime.strptime("2024-06-10", "%Y-%m-%d")
```

```
users = [
 {"username": "Amit", "isPremium": True,
 "lastLogin": "2024-06-08"},
 {"username": "Priya", "isPremium": False,
 "lastLogin": "2024-06-09"},
 {"username": "Rahul", "isPremium": True,
 "lastLogin": "2024-05-25"},
 {"username": "Neha", "isPremium": True,
 "lastLogin": "2024-06-05"}
]
```

```
def get_active_premium_users(users):
 result = []
```

```
for user in users:
 login_date = datetime.strptime(user["lastLogin"],
 "%Y-%m-%d")
 days = (today - login_date).days

 if user["isPremium"] and days <= 7:
 result.append(user)

return result

print(get_active_premium_users(users))
'''
```

### Expected Output

```
```python  
[  
{'username': 'Amit', 'isPremium': True, 'lastLogin':  
    '2024-06-08'},  
{'username': 'Neha', 'isPremium': True, 'lastLogin':  
    '2024-06-05'}  
]  
```
```

### *### Explanation:*

- \* The program checks whether the user has a Premium account.*
- \* It calculates how many days have passed since the last login.*
- \* Only Premium users who logged in within the last 7 days are returned.*

---

*## 5. Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.*

*### Answer:*

| <i>  Access Control Mistake</i>                      | <i>  Fix /</i>           |
|------------------------------------------------------|--------------------------|
| <i>Prevention Tip</i>                                | <i> </i>                 |
| <i>  -----  </i>                                     | <i>  -----</i>           |
| <i>-----  </i>                                       | <i> </i>                 |
| <i>  1. Weak passwords allowed</i>                   | <i>&gt;   Enforce</i>    |
| <i>strong password rules and minimum length.</i>     |                          |
| <i>  2. No role-based access control</i>             | <i>&gt;   Give users</i> |
| <i>only the permissions required for their role.</i> |                          |

- | 3. Sensitive pages accessible without login > | Require authentication before allowing access to protected pages.
- | 4. Sessions never expire > | Automatically log users out after a period of inactivity.
- | 5. No two-factor authentication > | Enable two-factor authentication (2FA) for extra account security.

#### ### Explanation:

*These mistakes can allow attackers or unauthorized users to access sensitive information. Using strong authentication, proper access control, secure session management, and 2FA helps protect user accounts and application data.*